Graph Neural Network Surrogates for a Coupled Heat-Conduction and Pyrolysis Solver in Thermal Protection System Design

Grey Goodwin
University of Kentucky
Lexington, KY, USA
grey.goodwin@uky.edu

***Abstract*—Thermal protection system (TPS) design relies on repeated evaluation of an expensive, coupled physics solver that resolves heat conduction, pyrolysis, pyrolysis-gas transport, and aerothermal surface chemistry. We present early work toward a graph neural network (GNN) surrogate that learns the per-step evolution of a 1D heat-shield pyrolysis solver. As a foundation, we re-implement a trusted Fortran reference solver in modular, vectorized Python and verify it against the original by column-level comparison of its thermocouple and density outputs, matching to machine precision on a conduction–pyrolysis case and to 0.064 K on a full gas and aerothermal case. Training on data from the validated solver, a per-cell message-passing GNN reproduces a held-out heating scenario over a full 60 s rollout to a mean error of 7.6 K and tracks the pyrolysis front to within a fraction of a millimeter. On the full gas case a direct per-cell predictor diverges on a held-out forcing, whereas a conservative flux-form variant that predicts face fluxes removes the divergence and holds a bounded rollout.**

***Index Terms*—graph neural networks, surrogate models, thermal protection systems, pyrolysis, scientific machine learning**

I. Introduction

Thermal protection systems (TPS) shield a vehicle from the intense aerothermal heating of atmospheric entry. Designing one means choosing a material and a thickness that keep the structure beneath the heat shield below its temperature limit while adding as little mass as possible. The governing physics is a coupled, strongly nonlinear problem: heat conducts into the material, the material pyrolyzes (chemically decomposes) as it heats, the pyrolysis gas percolates out through the charring solid, and the exposed surface exchanges energy with the hot boundary layer.

A high-fidelity solver that captures this coupling is expensive to run, and TPS design needs many such runs: parameter sweeps, sizing studies, and optimization loops each call the solver hundreds or thousands of times [1]–[3]. That cost is the bottleneck this work targets. The response we pursue is a machine-learned surrogate trained on data from a trusted reference solver, so that design loops can replace most solver calls with fast surrogate evaluations [4].

We use a graph neural network for the surrogate because the discretized solver is naturally a graph: the mesh cells are nodes and adjacent cells are connected by edges [5]. A GNN that shares its weights across all cells is mesh-native and, in principle, applies to a mesh of any size [6], the property we ultimately want for a tool that must run at varying resolutions.

This paper reports two things. First, a faithful Python re-implementation of the reference Fortran solver, verified against the original to machine precision on the conduction–pyrolysis case and to a small fraction of a kelvin on the full multi-physics case. Second, a first working surrogate trained on that validated solver's data, accurate over a full rollout on a held-out scenario, together with a diagnosed failure mode on the gas case and a conservative flux-form formulation that removes it.

II. Reference Solver

The reference is a 1D finite-volume heat-shield code written in Fortran. It models, in one spatial dimension through the thickness of the slab:

- **Heat conduction** with temperature-dependent and char-state-dependent material properties.
- **Pyrolysis**, the heat-driven decomposition of the solid, modeled as a set of Arrhenius reactions that consume reactive species and release gas.
- **Pyrolysis-gas transport** through the porous char (Darcy flow plus advection), which carries energy and couples back into the temperature field.
- **An aerothermal surface boundary**, including a B-prime surface-chemistry table, a blowing correction, and a wall-enthalpy balance, representing the interaction with the hot boundary layer.

The governing energy equation is the conduction-diffusion balance

$$\rho\, c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x}\left(k \frac{\partial T}{\partial x}\right) + (\text{pyrolysis and gas source terms}), \tag{1}$$

discretized on a finite-volume mesh. Each cell carries a temperature and a set of species densities; the conduction term is evaluated from temperature differences across cell faces, and the pyrolysis term advances each species by a closed-form Arrhenius update. Material properties are blended between virgin and char states by a char-progress variable derived from the local density.

Three reference cases ship with the code and are used throughout: `aw1`, a conduction-plus-pyrolysis case with a

prescribed wall-temperature history and the gas physics off; and `aw2_tc21` and `aw2_tc22`, which add the full gas and aerothermal boundary. As shipped, `aw2_tc22` is configured identically to `aw2_tc21`.

III. Methodology

A. Python Port

The solver was re-implemented in Python one physics module at a time: grid and geometry setup, mixture properties, the Arrhenius pyrolysis update, the gas solve, and the aerothermal boundary. Where a coding choice affected the numerical result, the Python deliberately matched the Fortran. The thermocouple sampling in particular was rewritten to mirror the reference probe-sampling routine, clamping to the face value at domain boundaries and reconstructing it by interpolation in the interior. A few details proved to matter for agreement: thermocouple positions are given as depth from the right wall; the face density used to evaluate the face conductivity is the pre-pyrolysis density, matching the Fortran update order; and output is written before the physics step, matching the reference time-offset convention.

B. Verification

Verification is done by direct column-by-column comparison against the Fortran output from a run with identical configuration. For every output column (time, each thermocouple temperature, and each density), we report the maximum absolute, mean absolute, and maximum relative error across the run. This is a strict test: it compares the full time history at every recorded probe, not just a summary statistic.

C. Training-Data Generation

Once verified, the Python solver generates the training data. Each run records the full per-cell field over time (temperature, bulk density, the per-species densities, and, in gas runs, the gas pressure and mass flux), including the two ghost cells at the domain ends that carry the boundary forcing. A single recorded trajectory can later be turned into training pairs at any time-step spacing without re-running the solver. The first dataset varies the boundary forcing, a sweep of wall-temperature histories, on the fast gas-off `aw1` case.

D. GNN Architecture

The surrogate is a per-cell message-passing network. Each finite-volume cell is a node carrying the state vector $[T, \rho, \rho_i, \text{porosity}]$. Adjacent cells are joined by edges whose features are relative: the difference between the neighbor's state and the cell's own, together with the cell spacing. This relative encoding mirrors the physics, since the conduction flux between two cells depends on their temperature difference and their spacing. A forward step encodes the node and edge features with small multilayer perceptrons, performs one round of message passing (each cell aggregates the encoded messages from its left and right neighbors and updates its hidden state), and decodes a predicted change in the cell's state, which is added to the current state and rolled forward in time. A single

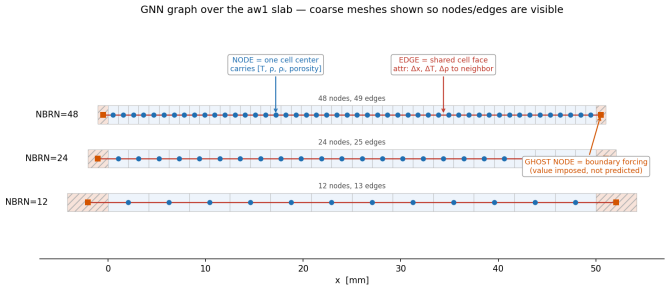

Fig. 1. The mesh as a graph: each finite-volume cell is a node carrying $[T, \rho, \rho_i, \text{porosity}]$; edges join neighboring cells with relative features (neighbor-minus-self state and the cell spacing Δx). Ghost nodes at the ends impose the boundary forcing. Weights are shared across all cells. (Coarse meshes shown for legibility.)

message-passing round matches the nearest-neighbor reach of the conduction stencil, and weight sharing across cells lets the same model apply to a mesh of any length. The model predicts only the independent degrees of freedom, temperature and the per-species densities; bulk density and porosity are derived from them. Training minimizes the error of a short multi-step rollout rather than a single step, which controls the accumulation of error over a long rollout (Section IV-B).

E. Conservative Flux-Form Variant

The gas-case failure (Section IV-C) motivates a second output formulation. Rather than predicting a per-cell state change directly, the model predicts a flux on each cell face and forms the change as the net flux across the cell's two faces plus a local source,

$$\Delta_i = \left(F_{i-1/2} - F_{i+1/2}\right) + S_i, \tag{2}$$

where $F_{i\pm1/2}$ are the face fluxes and S_i is a local source. The face flux is constructed antisymmetric, $F(a,b) = -F(b,a)$, giving two properties by construction: (i) conservation, since whatever leaves a cell across a face enters its neighbor; and (ii) a zero baseline, since a uniform field yields zero face flux, so quiescent cells receive no transport change and the per-step bias cannot arise. Change not driven by a neighbor gradient, the local pyrolysis source, is carried by S_i. The flux and source are produced by the same weight-shared message-passing network, and the change is decoded with no constant offset so the zero baseline survives. This is the conservative, divergence-form construction used for learned solvers [7], [8].

IV. Results

A. Port Validation

On `aw1` (conduction plus pyrolysis), all fifteen output columns match the Fortran output to approximately 10^{-8}, machine precision, including every interior thermocouple: the conduction and pyrolysis port is exact. On `aw2_tc21` (full gas plus aerothermal), with the gas physics enabled and compared against the same-configuration Fortran output, agreement is summarized in Table I.

TABLE I
PORT VALIDATION AGAINST THE FORTRAN REFERENCE (aw2_tc21).

Quantity	Agreement
All temperature columns	max 0.064 K, mean $\sim 4 \times 10^{-4}$ K
Hot aerothermal surface	0.06 K
Densities	$\sim 10^{-3}$ to 10^{-9}
Time, cold back face	bit-exact

TABLE II
HELD-OUT aw2 ROLLOUT: MEAN INTERIOR T ERROR, DIRECT VS. CONSERVATIVE FLUX-FORM.

	Direct	Flux-form
Over the full 120 s rollout	8755 K	121 K
At the end of the rollout	25434 K	104 K
End-of-rollout maximum	38359 K	312 K

During verification, an apparent early interior drift and a roughly 37.7 K back-face offset both proved to be artifacts of how the thermocouples were sampled, not numerical error; both vanished once the probe sampling matched the reference routine. Comparing two temperature fields through a sampling layer can manufacture a discrepancy that is not present in the physics itself. As shipped, aw2_tc22 is byte-for-byte identical in configuration to aw2_tc21 and so adds no independent validation condition.

B. Surrogate Accuracy (aw1)

Trained on solver-generated trajectories, the surrogate is rolled forward on its own predictions over a held-out aw1 heating scenario (gas off) for the full 60 s, with the known boundary forcing re-imposed each step. The mean rollout error is 7.6 K, roughly half a percent to one percent of the 298 to 1500 K field, and the surrogate tracks the char and pyrolysis front to within a fraction of a millimeter (Fig. 2). Reaching this required training on multi-step rollouts rather than single steps: a single-step-trained model is accurate for one prediction but accumulates error and drifts once rolled forward freely, and the end-of-trajectory error fell from several hundred kelvin to tens of kelvin under a multi-step rollout loss. This error-accumulation behavior, and its mitigation by noise injection and multi-step training, are well documented for learned simulators [5], [7].

C. Gas Case (aw2): A Diagnosed Failure Mode

Extending the surrogate to the full gas-plus-aerothermal case does not produce a usable model, and the way it fails is informative. Trained and rolled out on the same forcing it converges to a bounded error of about 625 K, but the proper test, training on a sweep of eight flux histories and rolling out on an unseen one, diverges to several thousand kelvin. The in-sample result is optimistic: the model largely fits the one trajectory it is evaluated on and does not generalize across forcings.

The cause is temporal, not spatial (Fig. 3). Per cell, aw2 is no steeper than aw1, since its finer mesh spreads the gradient over more cells. What differs is the rate of change per step: the aw2 surface, driven by a square-pulse aerothermal flux, heats from about 300 K to over 1500 K in roughly one second, so it moves hundreds of kelvin in a single coarse super-step against tens of kelvin for aw1, and a fixed super-step cannot track that. Making the model time-step aware and using an adaptive schedule, with fine steps near the transient and coarse steps on the plateau, converts the divergence into a bounded in-sample rollout (Fig. 4). A second, smaller effect is a per-step bias: the model predicts a small nonzero change for cold, untouched cells, which accumulates over a long rollout; a stay-put regularizer reduces it. Neither remedy closes the held-out gap, and we ruled out more training trajectories, treating the gas state as a known input, and a wider stencil. The reference solver was verified sound on this case, so the failure is the surrogate's, not the data's. This bounds where a per-cell state-delta GNN of this kind applies and motivates the conservative formulation of Section IV-D.

D. Conservative Flux-Form Removes the Divergence

The flux-form variant of the preceding section was trained on the same eight-forcing aw2 dataset as the direct model, with identical single-step training and the gas state held from truth so the test isolates the thermal and chemical rollout; the two models differ only in their output. Both were rolled out on a held-out forcing, not among the eight training cases, recorded at the training cadence so the comparison carries no time-step mismatch (Table II, Fig. 5).

The direct model diverges to $\sim$25,000 K by the end of the rollout, reproducing Section IV-C; the flux-form stays bounded near 100–160 K throughout, roughly a $240\times$ reduction in end error, on a forcing it did not see in training. The conservative formulation removes the divergence. Two qualifications: a bounded error near 100 K on a 300–1500 K field is stable but not yet accurate ($\sim$10%). The accuracy gap is not closed by multi-step rollout training: applied to the flux-form variant on the eight-forcing sweep it lowers the training loss but worsens the held-out rollout to several hundred kelvin, overfitting only eight stiff trajectories where the gas-off result used twenty-four. The remaining gap is therefore training-data quantity, not stability.

E. Wall Time

The Fortran solver runs aw2_tc21 in about 3 minutes; the present unoptimized Python implementation takes about 15.6 minutes, roughly five times slower, as expected for interpreted array code against compiled Fortran. The Python port's role is to generate verified training data and to be a readable reference, not to be fast; the speed argument of this program is the surrogate, whose wall-time advantage over the solver is the subject of ongoing measurement.

V. DISCUSSION

The results establish three things. First, the multi-physics reference solver can be reproduced faithfully in modular,

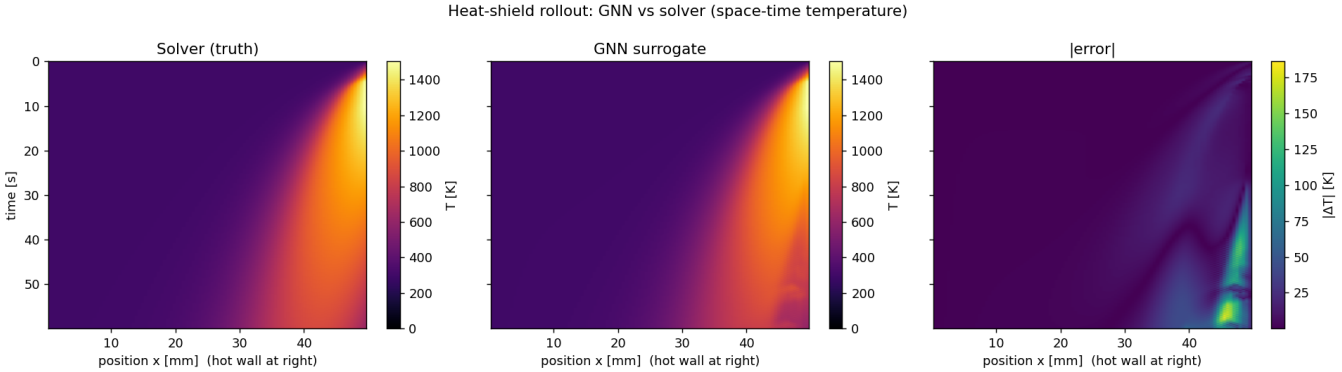

Fig. 2. aw1 surrogate rollout (gas off), space-time temperature: solver truth (left), GNN surrogate (center), absolute error (right). Mean rollout error 7.6 K over the full 60 s; the surrogate field is visually indistinguishable from the solver and the error is small everywhere.

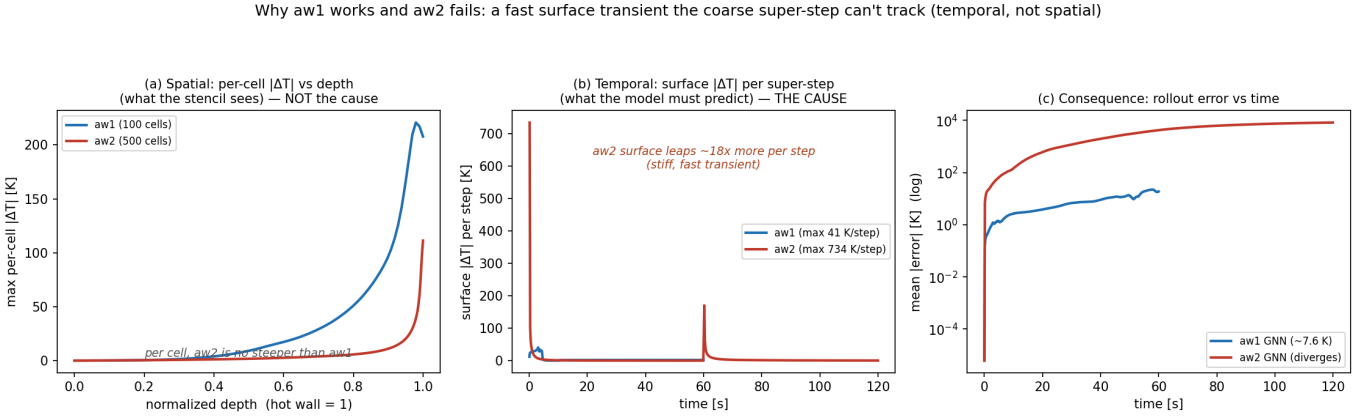

Fig. 3. Why aw2 fails and aw1 does not, and why the cause is temporal not spatial. (a) Per-cell spatial $|\Delta T|$: aw2 is no steeper than aw1. (b) Per-super-step surface $|\Delta T|$: aw2 leaps $\sim$18$\times$ more (734 vs 41 K) under the square-pulse aerothermal flux. (c) The consequence: aw1 rollout error stays $\sim$7.6 K while aw2 diverges.

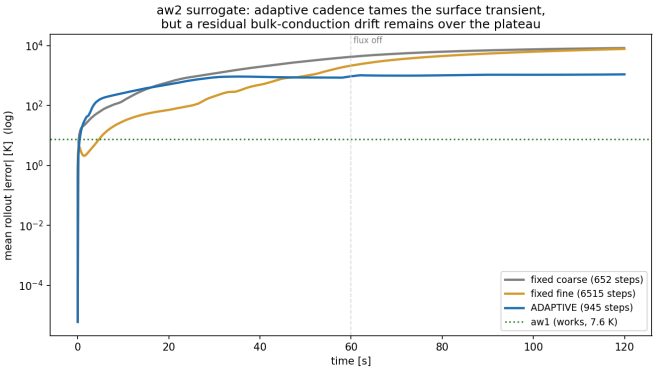

Fig. 4. aw2 rollout error vs time across recipes. Fixed coarse and fixed fine cadence both diverge; the Δt-aware adaptive schedule stays bounded. All shown in-sample; held-out, even the adaptive model does not yet generalize (Section IV-C).

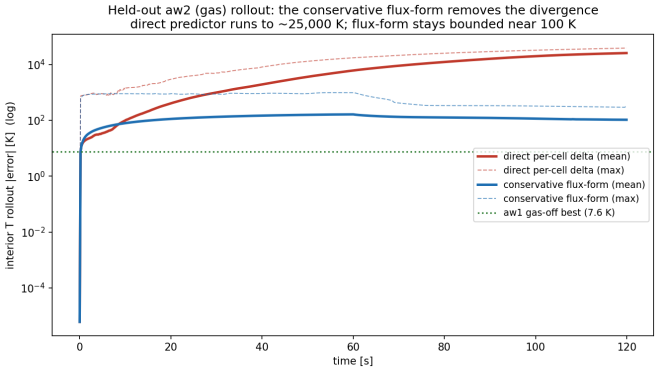

Fig. 5. Held-out `aw2` rollout error vs. time (log scale). The direct per-cell-delta model diverges past 10^4 K; the conservative flux-form stays bounded near 100 K on the unseen forcing.

readable Python: to machine precision on the conduction-plus-pyrolysis case and to a small fraction of a kelvin on the full gas case. That fidelity is what makes the solver usable as a data generator, since a surrogate is only as trustworthy as the data it learns from. Second, a per-cell message-passing GNN trained on that data is an accurate surrogate for the gas-off case over a full rollout, including faithful tracking of the moving pyrolysis front; its relative edge features give it the local information the physics uses, and its weight sharing is what makes it mesh-

native.

Third, the gas case marks the edge of where the direct architecture works, and the boundary is sharp and explainable (Section IV-C). The same model fails to generalize there because a fixed super-step cannot follow the stiff aerothermal surface transient and because a small per-step bias on quiescent cells accumulates over a long rollout. A per-cell state-delta predictor learns local dynamics well, but a fixed-step rollout is the wrong vehicle for a fast, localized forcing transient. The conservative flux-form, in which a uniform field produces exactly zero change by construction, removes the divergence (Section IV-D): on the same held-out forcing the direct model runs past 25,000 K while the flux-form stays bounded near 100 K. Making it accurate, rather than only stable, remains open. The verification caution of Section IV-B, that a sampling layer can mask or invent discrepancies, applies equally when comparing a surrogate field against a solver field.

VI. Future Work

The conservative flux-form already removes the gas-case divergence (Section IV-D); the remaining work is accuracy, not stability. Multi-step rollout training, which sharpened the gas-off case, overfits the present eight-forcing sweep, so the first step is a larger forcing sweep, then multi-step training, and a time-step-aware adaptive schedule for the stiff aerothermal transient. Beyond the gas case, training across a range of mesh resolutions would let one model run at resolutions it did not see; the surrogate's wall-time advantage over the solver should be measured across a design-relevant set of evaluations; and the physics can be extended with radiation, surface recession, and two- and three-dimensional meshes.

VII. Conclusion

We re-implemented a coupled heat-conduction and pyrolysis heat-shield solver in Python, verified it against the Fortran reference to machine precision on the gas-off case and to 0.064 K on the full gas case, and trained a per-cell message-passing GNN surrogate that reproduces a held-out gas-off rollout to 7.6 K. On the gas case the direct surrogate diverges, a failure we trace to a fixed-step state-delta formulation; a conservative flux-form variant that predicts face fluxes removes the divergence and yields a bounded, if not yet accurate, rollout on an unseen forcing. Multi-step training and adaptive time-stepping for the flux-form model, mesh-resolution generalization, and extension to higher dimensions are the next steps.

Acknowledgment

This work builds on the reference heat-shield solver developed by Dr. Martin and on a graph-neural-network surrogate program within the group.

References

[1] W. Pan and B. Gao, "Establishment and optimization of ablation surrogate model for thermal protection material," *Journal of Beijing Institute of Technology*, vol. 32, no. 4, pp. 477–493, 2023.

[2] A. Mazzaracchio and M. Marchetti, "A probabilistic sizing tool and monte carlo analysis for entry vehicle ablative thermal protection systems," *Acta Astronautica*, vol. 66, no. 5–6, pp. 821–835, 2010.

[3] G. Lu, Z. Shi, R. Zhang, Y. Li, and K. Zhang, "Probabilistic design and optimization of thermal protection system with variable thickness based on non-uniform aerodynamic heating," *International Journal of Heat and Mass Transfer*, vol. 225, p. 125386, 2024.

[4] N. V. Queipo, R. T. Haftka, W. Shyy, T. Goel, R. Vaidyanathan, and P. K. Tucker, "Surrogate-based analysis and optimization," *Progress in Aerospace Sciences*, vol. 41, no. 1, pp. 1–28, 2005.

[5] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia, "Learning to simulate complex physics with graph networks," in *International Conference on Machine Learning (ICML)*, 2020.

[6] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, "Learning mesh-based simulation with graph networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[7] J. Brandstetter, D. E. Worrall, and M. Welling, "Message passing neural PDE solvers," in *International Conference on Learning Representations (ICLR)*, 2022.

[8] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, p. e2101784118, 2021.